

J.A.R.V.I.S Project_Review Report

Project: CyberShield AI

Generated: 2026-07-02 17:53:43

J.A.R.V.I.S ENTERPRISE PROJECT REVIEW

Generated: 2026-07-02 17:52:59

Project: CyberShield AI

Mode: safe analysis / no automatic code changes

Every recommendation must be verified with tests before applying changes.

1. PROJECT SCORECARD

Project: CyberShield_AI_Enterprise_Hardened_Enhanced

Path: E:\stick\De facut\Cyber Security App\CyberShield_AI_Enterprise_Hardened_Enhanced

Files indexed: 78

Tech stack: CSS, Docker, Express, FastAPI, Flask, HTML, JWT, JavaScript, Node.js, Pytest, Python, React, SQLAlchemy, SQLite, Tailwind, TypeScript, Vite

Scores:

- Security: 5.0/10
- Architecture: 7.2/10
- Maintainability: 7.0/10
- Overall: 6.4/10

Reasons:

Security:

- Authentication/authorization files detected.
- JWT/token handling evidence detected.
- Password hashing evidence detected.
- Environment/config/secret handling evidence detected.
- Tests detected.
- CI/workflow files detected.
- Upload/file handling evidence detected.
- Weak/demo secret or credential strings detected: admin\admin.py -> admin, admin\admin.py -> secret, api.py -> admin, api.py -> password, api.py -> secret, auditUI.py -> admin, auditUI.py -> changeme, auditUI.py -> changeme123

Architecture:

- Backend framework evidence detected.
- Frontend/UI layer detected.
- Database/storage layer detected.
- Backend routes detected.
- Frontend API calls detected.
- Docker/deployment evidence detected.
- Dependency manifests detected.
- Both FastAPI and Flask evidence detected; verify if this is intentional.

Maintainability:

- Tests detected.
- Dependency manifests detected.
- CI/workflow files detected.
- Docker/deployment files detected.
- Frontend source organization detected.
- Python source files detected.
- Non-trivial project size detected.

Evidence summary:

- Backend routes: 17
- Frontend API calls: 1
- Auth files: 15
- JWT files: 11
- Frontend files: 8
- Test files: 6
- Docker files: 1
- CI files: 1

2. STRICT GROUNDED ANALYZER

STRICT GROUNDED ANALYZER REPORT

Mode: rule-based / no LLM / no speculation

Project: CyberShield_AI_Enterprise_Hardened_Enhanced

Path: E:\stick\De facut\Cyber Security App\CyberShield_AI_Enterprise_Hardened_Enhanced

Files indexed: 78

Detected tech stack from deep memory: CSS, Docker, Express, FastAPI, Flask, HTML, JWT, JavaScript, Node.js, Pytest, Python, React, SQLAlchemy, SQLite, Tailwind, TypeScript, Vite

Extensions: {'.py': 48, '.json': 5, '.md': 6, '.sh': 3, '.yaml': 2, '.txt': 1, '.html': 4, '.sql': 1, '.tsx': 4, '.js': 2, '.ts': 1, '.css': 1}

Dependency evidence:

- package.json: {'dependencies': ['@tailwindcss/forms', '@tailwindcss/typography', 'axios', 'cssnano', 'postcss-nesting', 'postcss-preset-env', 'react', 'react-dom'], 'devDependencies': ['@types/axios', '@types/react', '@types/react-dom', '@vitejs/plugin-react', 'autoprefixer', 'postcss', 'tailwindcss', 'typescript', 'vite'], 'scripts': {'dev': 'vite', 'build': 'vite build', 'preview': 'vite preview'}}
- file-scan-log-app\package.json: {'dependencies': ['axios', 'react', 'react-dom'], 'devDependencies': ['@vitejs/plugin-react', 'eslint', 'prettier', 'typescript', 'vite', 'vitest'], 'scripts': {'dev': 'vite', 'build': 'vite build', 'preview': 'vite preview', 'lint': 'eslint . --ext .ts,.tsx --fix', 'format': 'prettier --write .', 'audit': 'npm audit', 'test': 'vitest'}}
- requirements.txt: ['flask', 'bcrypt', 'pyotp', 'psycpg2-binary', 'boto3', 'python-dotenv']

FastAPI evidence files:

- auditUI.py
- routes\admin.py
- routes\auth.py
- routes\dashboard.py
- routes__init__.py
- test\test_auth.py
- tests\test_auth.py
- utils\auth.py

Flask evidence files:

- middleware.py
- security.py
- __init__.py
- admin\admin.py
- dashboard\dashboard.py
- templates\Dashboard Audit\audit_dashboard.py
- tests\test_dashboard_access.py

Authentication / authorization related files:

- auth.py
- permissions.py
- permissions_dynamic.py
- admin\admin.py
- admin\admin_permissions.html
- routes\admin.py
- routes\auth.py
- schemas\admin.py
- schemas\auth.py
- test\test_auth.py
- tests\test_auth.py
- tests\test_permissions.py
- utils\auth.py
- utils\auth_utils.py
- utils\token.py

JWT / token related files:

- config.py
- middleware.py
- docs\SECURITY_OVERVIEW.md
- file-scan-log-app\src\FileScanLog.tsx
- routes\auth.py
- schemas\auth.py
- test\test_auth.py
- tests\test_auth.py
- utils\auth.py
- utils\auth_utils.py
- utils\token.py

Password hashing evidence files:

- api.py
- encryptor.py
- requirements.txt
- routes\auth.py

Database / storage evidence files:

- api.py
- audit_log.py
- auth.py
- config.py
- db.py
- encryptor.py
- package-lock.json
- permissions_dynamic.py
- scanner.py
- security.py
- admin\admin.py
- audit_backups\export_audit_secure.py
- backend\audit_log.py
- backend\backup.py
- backend\database\init_db.py
- dashboard\dashboard.py
- docs\DEPLOYMENT_GUIDE.md
- file-scan-log-app\package-lock.json
- models\user.py
- routes\admin.py
- routes\auth.py
- routes\dashboard.py
- templates\Dashboard Audit\audit_dashboard.py
- tests\test_dashboard_access.py
- tests\test_permissions.py
- utils\auth.py
- utils\logger.py

Frontend / UI evidence files:

- components\AppLayout.tsx
- file-scan-log-app\postcss.config.js
- file-scan-log-app\tailwind.config.js
- file-scan-log-app\vite.config.ts
- file-scan-log-app\src\FileScanLog.tsx
- file-scan-log-app\src\index.css
- file-scan-log-app\src\main.tsx
- file-scan-log-app\src\ScanForm.tsx

File upload / file scan evidence files:

- api.py
- auditUI.py
- audit_log.py
- backup.py
- db.py
- encryptor.py
- package-lock.json
- package.json
- permissions.py
- scanner.py
- .github\workflows\security.yml
- audit_backups\decrypt.py
- audit_backups\export_audit.py
- audit_backups\export_audit_secure.py
- backend\backup.py
- backend\database\file_scans.sql
- backend\database\init_db.py
- backend\utils\crypto.py
- crypto\crypto.py
- crypto\key_rotation_scheduler.py
- docs\DEPLOYMENT_GUIDE.md
- file-scan-log-app\package-lock.json
- file-scan-log-app\package.json
- file-scan-log-app\src\FileScanLog.tsx
- file-scan-log-app\src\index.css
- file-scan-log-app\src\main.tsx
- file-scan-log-app\src\ScanForm.tsx

Logging / audit evidence files:

- api.py
- auditUI.py
- audit_log.py
- auth.py
- backup.py
- db.py

- email_alert.py
- encryptor.py
- explain.py
- gui.py
- logger.py
- permissions.py
- permissions_dynamic.py
- scanner.py
- security.py
- __init__.py
- admin\admin.py
- audit_backups\export_audit.py
- audit_backups\export_audit_secure.py
- backend\audit_log.py
- backend\backup.py
- backend\database\file_scans.sql
- backend\database\init_db.py
- backend\utils\crypto.py
- crypto\key_rotation_scheduler.py
- dashboard\dashboard.py
- file-scan-log-app\package.json
- routes\admin.py
- routes\auth.py
- routes\dashboard.py
- ... and 2 more

Docker / deployment evidence files:

- docker-compose.yml

CI / workflow evidence files:

- .github\workflows\security.yml

Test evidence files:

- test\security_scan.sh
- test\test_auth.py
- test\test_crypto.py
- tests\test_auth.py
- tests\test_dashboard_access.py
- tests\test_permissions.py

Detected backend routes:

- auditUI.py -> GET /audit-ui
- auditUI.py -> POST /audit-ui/delete
- auditUI.py -> GET /audit-ui/export
- security.py -> /login
- admin\admin.py -> ROUTE /admin
- admin\admin.py -> ROUTE /admin/grant
- admin\admin.py -> ROUTE /admin/revoke
- admin\admin.py -> /admin
- admin\admin.py -> /admin/grant
- admin\admin.py -> /admin/revoke
- dashboard\dashboard.py -> /dashboard
- routes\admin.py -> GET /dashboard
- routes\admin.py -> PUT /update
- routes\auth.py -> POST /login
- routes\dashboard.py -> GET /
- templates\Dashboard Audit\audit_dashboard.py -> ROUTE /audit
- templates\Dashboard Audit\audit_dashboard.py -> /audit

Detected frontend API calls:

- file-scan-log-app\src\ScanForm.tsx -> POST /api/scan

Conclusion:

This report only lists evidence found directly in indexed files. It does not infer hidden services, cloud infrastructure, ML models, or databases unless present in files.

3. STRICT SECURITY ANALYZER

STRICT SECURITY ANALYZER REPORT

Mode: rule-based / no LLM / no speculation

False-positive reduction: enabled

Project: CyberShield_AI_Enterprise_Hardened_Enhanced

Files indexed: 78

Security findings:

- HIGH: admin\admin.py contains weak/default secret or demo credential string: admin
- HIGH: admin\admin.py contains weak/default secret or demo credential string: secret
- HIGH: api.py contains weak/default secret or demo credential string: admin
- HIGH: api.py contains weak/default secret or demo credential string: password
- HIGH: api.py contains weak/default secret or demo credential string: secret
- HIGH: audit_backups\export_audit_secure.py contains weak/default secret or demo credential string: admin
- HIGH: audit_backups\export_audit_secure.py contains weak/default secret or demo credential string: secret
- HIGH: auditUI.py contains weak/default secret or demo credential string: admin
- HIGH: auditUI.py contains weak/default secret or demo credential string: changeme
- HIGH: auditUI.py contains weak/default secret or demo credential string: changeme123
- HIGH: auditUI.py contains weak/default secret or demo credential string: dev
- HIGH: backend\utils\crypto.py contains weak/default secret or demo credential string: secret
- HIGH: backup.py contains weak/default secret or demo credential string: secret
- HIGH: config.py contains weak/default secret or demo credential string: password
- HIGH: config.py contains weak/default secret or demo credential string: secret
- HIGH: crypto\crypto.py contains weak/default secret or demo credential string: secret
- HIGH: crypto\key_rotation_scheduler.py contains weak/default secret or demo credential string: secret
- HIGH: crypto\key_rotation_scheduler.py contains weak/default secret or demo credential string: test
- HIGH: docs\DEPLOYMENT_GUIDE.md contains weak/default secret or demo credential string: secret
- HIGH: encryptor.py contains weak/default secret or demo credential string: admin
- HIGH: encryptor.py contains weak/default secret or demo credential string: password
- HIGH: encryptor.py contains weak/default secret or demo credential string: secret
- HIGH: middleware.py contains JWT handling without obvious expiration evidence.
- HIGH: routes\auth.py contains fake_users_db / hardcoded demo users. Replace with real database-backed users before production.
- HIGH: routes\auth.py contains weak/demo password strings. Remove demo credentials from source code.
- HIGH: utils\auth.py contains JWT handling without obvious expiration evidence.
- HIGH: utils\auth.py contains weak/default secret or demo credential string: admin
- HIGH: utils\auth.py contains weak/default secret or demo credential string: password
- HIGH: utils\auth.py contains weak/default secret or demo credential string: secret
- HIGH: utils\auth_utils.py contains weak/default secret or demo credential string: secret
- HIGH: utils\auth_utils.py contains weak/default secret or demo credential string: your-secret-key
- HIGH: utils\token.py contains weak/default secret or demo credential string: secret
- MEDIUM: __init__.py uses print-based audit/security logging. Consider structured logging with retention.
- MEDIUM: admin\admin.py reads security-sensitive environment variables, but missing-value validation is not obvious.
- MEDIUM: aes_cipher.py reads security-sensitive environment variables, but missing-value validation is not obvious.
- MEDIUM: api.py reads security-sensitive environment variables, but missing-value validation is not obvious.
- MEDIUM: audit_backups\export_audit.py references uploaded filenames without obvious filename sanitization.
- MEDIUM: audit_backups\export_audit.py uses print-based audit/security logging. Consider structured logging with retention.
- MEDIUM: audit_backups\export_audit_secure.py reads security-sensitive environment variables, but missing-value validation is not obvious.
- MEDIUM: audit_backups\export_audit_secure.py references uploaded filenames without obvious filename sanitization.
- MEDIUM: backend\database\init_db.py references uploaded filenames without obvious filename sanitization.
- MEDIUM: backend\database\init_db.py uses print-based audit/security logging. Consider structured logging with retention.
- MEDIUM: backup.py reads security-sensitive environment variables, but missing-value validation is not obvious.
- MEDIUM: dashboard\dashboard.py uses print-based audit/security logging. Consider structured logging with retention.
- MEDIUM: db.py references uploaded filenames without obvious filename sanitization.
- MEDIUM: file-scan-log-app\src\FileScanLog.tsx references uploaded filenames without obvious filename sanitization.
- MEDIUM: scanner.py references uploaded filenames without obvious filename sanitization.
- MEDIUM: security.py uses print-based audit/security logging. Consider structured logging with retention.
- MEDIUM: utils\logger.py uses print-based audit/security logging. Consider structured logging with retention.
- LOW: auth.py logs username/IP style data. Review privacy and retention requirements.
- LOW: backend\database\init_db.py logs username/IP style data. Review privacy and retention requirements.
- LOW: db.py logs username/IP style data. Review privacy and retention requirements.
- LOW: email_alert.py logs username/IP style data. Review privacy and retention requirements.
- LOW: routes\admin.py logs username/IP style data. Review privacy and retention requirements.
- LOW: routes\auth.py logs username/IP style data. Review privacy and retention requirements.
- LOW: routes\dashboard.py logs username/IP style data. Review privacy and retention requirements.
- LOW: security.py logs username/IP style data. Review privacy and retention requirements.
- LOW: utils\logger.py logs username/IP style data. Review privacy and retention requirements.
- POSITIVE: admin\admin.py reads security-sensitive configuration from environment variables.
- POSITIVE: aes_cipher.py reads security-sensitive configuration from environment variables.
- POSITIVE: api.py contains file size validation.
- POSITIVE: api.py contains SQLAlchemy ORM evidence.
- POSITIVE: api.py reads security-sensitive configuration from environment variables.
- POSITIVE: audit_backups\export_audit_secure.py contains SQLAlchemy ORM evidence.
- POSITIVE: audit_backups\export_audit_secure.py reads security-sensitive configuration from environment variables.

- POSITIVE: auditUI.py reads security-sensitive configuration from environment variables.
- POSITIVE: backend\utils\crypto.py reads security-sensitive configuration from environment variables.
- POSITIVE: backup.py reads security-sensitive configuration from environment variables.
- POSITIVE: config.py appears to include token expiration handling.
- POSITIVE: crypto\crypto.py reads security-sensitive configuration from environment variables.
- POSITIVE: encryptor.py contains file size validation.
- POSITIVE: encryptor.py contains SQLAlchemy ORM evidence.
- POSITIVE: encryptor.py reads security-sensitive configuration from environment variables.
- POSITIVE: file-scan-log-app\src\FileScanLog.tsx appears to include token expiration handling.
- POSITIVE: file-scan-log-app\src\ScanForm.tsx contains file size validation.
- POSITIVE: file-scan-log-app\src\ScanForm.tsx contains file type validation.
- POSITIVE: logger.py reads security-sensitive configuration from environment variables.
- POSITIVE: routes\auth.py appears to include token expiration handling.
- POSITIVE: routes\auth.py contains password hashing evidence.
- POSITIVE: routes\auth.py uses OAuth2PasswordRequestForm-style login handling.
- POSITIVE: utils\auth_utils.py appears to include token expiration handling.
- POSITIVE: utils\auth_utils.py reads security-sensitive configuration from environment variables.
- POSITIVE: utils\token.py appears to include token expiration handling.
- INFO: api.py contains SQLite evidence.
- INFO: backend\database\init_db.py contains SQLite evidence.
- INFO: config.py contains JWT/token handling evidence.
- INFO: docs\SECURITY_OVERVIEW.md contains JWT/token handling evidence.
- INFO: encryptor.py contains SQLite evidence.
- INFO: file-scan-log-app\src\FileScanLog.tsx contains JWT/token handling evidence.
- INFO: file-scan-log-app\src\ScanForm.tsx contains upload/form-data logic. Confirm backend validation exists.
- INFO: middleware.py contains JWT/token handling evidence.
- INFO: routes\auth.py contains JWT/token handling evidence.
- INFO: utils\auth.py contains JWT/token handling evidence.
- INFO: utils\auth_utils.py contains JWT/token handling evidence.
- INFO: utils\token.py contains JWT/token handling evidence.

Files to inspect first:

- auth.py
- permissions.py
- permissions_dynamic.py
- admin\admin.py
- admin\admin_permissions.html
- routes\admin.py
- routes\auth.py
- schemas\admin.py
- schemas\auth.py
- test\test_auth.py
- tests\test_auth.py
- tests\test_permissions.py
- utils\auth.py
- utils\auth_utils.py
- utils\token.py
- config.py
- middleware.py
- docs\SECURITY_OVERVIEW.md
- file-scan-log-app\src\FileScanLog.tsx
- routes\auth.py
- schemas\auth.py
- test\test_auth.py
- tests\test_auth.py
- utils\auth.py
- utils\auth_utils.py
- utils\token.py
- aes_cipher.py
- api.py
- auditUI.py
- backup.py
- config.py
- encryptor.py
- explain.py
- logger.py
- admin\admin.py
- audit_backups\decrypt.py
- audit_backups\export_audit.py
- audit_backups\export_audit_secure.py
- backend\backup.py
- backend\utils\crypto.py
- ... and 70 more

Recommended manual checks:

- Confirm production secrets are not default/demo values.
- Ignore findings in docs/memory/reports unless they contain real active credentials.
- Confirm authentication uses persistent users, not fake/demo dictionaries.
- Confirm JWT signing key is strong and stored outside source code.
- Confirm upload backend validates file type, size, filename, and content.
- Confirm logs do not expose passwords, tokens, secrets, or unnecessary personal data.

4. STRICT ARCHITECTURE ANALYZER

STRICT ARCHITECTURE ANALYZER REPORT

Mode: rule-based / no LLM / no speculation

Project: CyberShield_AI_Enterprise_Hardened_Enhanced

Path: E:\stickDe facut\Cyber Security App\CyberShield_AI_Enterprise_Hardened_Enhanced

Architecture evidence summary:

- FastAPI evidence detected.
- Flask evidence detected.
- Frontend/UI evidence detected.
- Database/storage evidence detected.
- Docker/deployment evidence detected.

Backend/API files:

- auditUI.py
- routes\admin.py
- routes\auth.py
- routes\dashboard.py
- routes__init__.py
- test\test_auth.py
- tests\test_auth.py
- utils\auth.py
- middleware.py
- security.py
- __init__.py
- admin\admin.py
- dashboard\dashboard.py
- templates\Dashboard Audit\audit_dashboard.py
- tests\test_dashboard_access.py
- auth.py
- permissions.py
- permissions_dynamic.py
- admin\admin.py
- admin\admin_permissions.html
- routes\admin.py
- routes\auth.py
- schemas\admin.py
- schemas\auth.py
- test\test_auth.py
- tests\test_auth.py
- tests\test_permissions.py
- utils\auth.py
- utils\auth_utils.py
- utils\token.py

Frontend files:

- components\AppLayout.tsx
- file-scan-log-app\postcss.config.js
- file-scan-log-app\tailwind.config.js
- file-scan-log-app\vite.config.ts
- file-scan-log-app\src\FileScanLog.tsx
- file-scan-log-app\src\index.css
- file-scan-log-app\src\main.tsx
- file-scan-log-app\src\ScanForm.tsx

Database/config files:

- api.py
- audit_log.py
- auth.py
- config.py
- db.py
- encryptor.py
- package-lock.json
- permissions_dynamic.py
- scanner.py

- security.py
- admin\admin.py
- audit_backups\export_audit_secure.py
- backend\audit_log.py
- backend\backup.py
- backend\database\init_db.py
- dashboard\dashboard.py
- docs\DEPLOYMENT_GUIDE.md
- file-scan-log-app\package-lock.json
- models\user.py
- routes\admin.py
- routes\auth.py
- routes\dashboard.py
- templates\Dashboard Audit\audit_dashboard.py
- tests\test_dashboard_access.py
- tests\test_permissions.py
- utils\auth.py
- utils\logger.py
- config.py
- docker-compose.yml
- package.json
- requirements.txt
- file-scan-log-app\package.json
- file-scan-log-app\postcss.config.js
- file-scan-log-app\tailwind.config.js
- file-scan-log-app\vite.config.ts

Deployment/CI files:

- docker-compose.yml
- .github\workflows\security.yml

Routes detected:

- auditUI.py -> GET /audit-ui
- auditUI.py -> POST /audit-ui/delete
- auditUI.py -> GET /audit-ui/export
- security.py -> /login
- admin\admin.py -> ROUTE /admin
- admin\admin.py -> ROUTE /admin/grant
- admin\admin.py -> ROUTE /admin/revoke
- admin\admin.py -> /admin
- admin\admin.py -> /admin/grant
- admin\admin.py -> /admin/revoke
- dashboard\dashboard.py -> /dashboard
- routes\admin.py -> GET /dashboard
- routes\admin.py -> PUT /update
- routes\auth.py -> POST /login
- routes\dashboard.py -> GET /
- templates\Dashboard Audit\audit_dashboard.py -> ROUTE /audit
- templates\Dashboard Audit\audit_dashboard.py -> /audit

Frontend API calls detected:

- file-scan-log-app\src\ScanForm.tsx -> POST /api/scan

Architecture gaps detected by rules:

- Check whether frontend API base URLs are centralized.
- Check whether FastAPI/Flask structure is intentionally mixed or legacy.
- Check whether config/secrets are consistently environment-based.
- Check whether tests cover routes, permissions, upload, token generation, and dashboard access.

5. DEAD CODE SCAN

Dead code / unused code scan

Important: These are heuristic results, not delete commands.

Production code is separated from tests and documentation/config files.

Possible unused production files:

aes_cipher.py [MEDIUM RISK - verify references and runtime usage first]
auditUI.py [MEDIUM RISK - verify references and runtime usage first]

Possible unused test files:

test\security_scan.sh [LOW RISK TO DELETE / archive only if tests are obsolete]
test\test_crypto.py [LOW RISK TO DELETE / archive only if tests are obsolete]
tests\test_dashboard_access.py [LOW RISK TO DELETE / archive only if tests are obsolete]

Possible unused documentation/config/deployment files:

docs\DEPLOYMENT_GUIDE.md [MEDIUM RISK - documentation/config/deployment may be used outside code]
docs\SECURITY_OVERVIEW.md [MEDIUM RISK - documentation/config/deployment may be used outside code]
file-scan-log-app\postcss.config.js [MEDIUM RISK - documentation/config/deployment may be used outside code]
file-scan-log-app\tailwind.config.js [MEDIUM RISK - documentation/config/deployment may be used outside code]

Possible unused production functions/components:

auth.py -> function request_password_reset [MEDIUM RISK - verify references before removing]
auth.py -> function verify_2fa_code [MEDIUM RISK - verify references before removing]
auth.py -> function generate_2fa_secret [MEDIUM RISK - verify references before removing]
backup.py -> function upload_backup_to_s3 [MEDIUM RISK - verify references before removing]
gui.py -> function run_explainability [MEDIUM RISK - verify references before removing]
permissions.py -> function list_permissions [MEDIUM RISK - verify references before removing]
__init__.py -> function create_app [HIGH RISK - may be called by framework/router/dynamic import]
__init__.py -> function handle_exception [HIGH RISK - may be called by framework/router/dynamic import]
admin\admin.py -> function admin_grant [MEDIUM RISK - verify references before removing]
admin\admin.py -> function admin_revoke [MEDIUM RISK - verify references before removing]
backend\utils\crypto.py -> function loop_encrypt [MEDIUM RISK - verify references before removing]
backend\utils\crypto.py -> function loop_decrypt [MEDIUM RISK - verify references before removing]
crypto\crypto.py -> function from_env [MEDIUM RISK - verify references before removing]
crypto\crypto.py -> function decrypt_file [MEDIUM RISK - verify references before removing]
routes\admin.py -> function get_admin_dashboard [HIGH RISK - may be called by framework/router/dynamic import]
routes\admin.py -> function update_admin_settings [HIGH RISK - may be called by framework/router/dynamic import]
routes\dashboard.py -> function get_dashboard_data [HIGH RISK - may be called by framework/router/dynamic import]

Possible unused test functions:

test\test_auth.py -> function test_login_valid_user [LOW RISK - test function, used by test runner]
test\test_auth.py -> function test_login_invalid_user [LOW RISK - test function, used by test runner]
test\test_auth.py -> function test_reset_password_flow [LOW RISK - test function, used by test runner]
test\test_crypto.py -> function test_encryption_decryption [LOW RISK - test function, used by test runner]
test\test_crypto.py -> function test_multiple_encryptions_different_results [LOW RISK - test function, used by test runner]
test\test_crypto.py -> function test_empty_string [LOW RISK - test function, used by test runner]
test\test_crypto.py -> function test_unicode_and_symbols [LOW RISK - test function, used by test runner]
test\test_crypto.py -> function test_decrypt_with_wrong_key [LOW RISK - test function, used by test runner]
tests\test_auth.py -> function test_login_success [LOW RISK - test function, used by test runner]
tests\test_auth.py -> function test_login_invalid_password [LOW RISK - test function, used by test runner]
tests\test_auth.py -> function test_login_with_unknown_ip [LOW RISK - test function, used by test runner]
tests\test_auth.py -> function test_login_with_2fa_enabled [LOW RISK - test function, used by test runner]
tests\test_dashboard_access.py -> function setup_test_permissions [LOW RISK - test function, used by test runner]
tests\test_dashboard_access.py -> function test_dashboard_view_for_analyst [LOW RISK - test function, used by test runner]
tests\test_dashboard_access.py -> function test_dashboard_view_for_guest [LOW RISK - test function, used by test runner]
tests\test_permissions.py -> function setup_module [LOW RISK - test function, used by test runner]
tests\test_permissions.py -> function teardown_module [LOW RISK - test function, used by test runner]
tests\test_permissions.py -> function test_grant_and_check_permission [LOW RISK - test function, used by test runner]
tests\test_permissions.py -> function test_revoke_and_check_permission [LOW RISK - test function, used by test runner]

Possible unused production classes:

logger.py -> class SlidingWindowLimiter [MEDIUM RISK - verify references before removing]
logger.py -> class TokenBucketLimiter [MEDIUM RISK - verify references before removing]
logger.py -> class ExponentialBackoffLimiter [MEDIUM RISK - verify references before removing]
models\user.py -> class UserCreate [HIGH RISK - may be called by framework/router/dynamic import]
models\user.py -> class UserOut [HIGH RISK - may be called by framework/router/dynamic import]
models\user.py -> class UserInDB [HIGH RISK - may be called by framework/router/dynamic import]
schemas\auth.py -> class UserLoginResponse [HIGH RISK - may be called by framework/router/dynamic import]

Possible unused test classes:

None found.

Possible unused production imports:

aes_cipher.py -> import Crypto.Random [MEDIUM RISK - verify references before removing]
aes_cipher.py -> import Crypto.Util.Padding [MEDIUM RISK - verify references before removing]
aes_cipher.py -> import dotenv [MEDIUM RISK - verify references before removing]
api.py -> import hashlib [HIGH RISK - may be called by framework/router/dynamic import]
api.py -> import bcrypt [HIGH RISK - may be called by framework/router/dynamic import]
api.py -> import hmac [HIGH RISK - may be called by framework/router/dynamic import]
api.py -> import secrets [HIGH RISK - may be called by framework/router/dynamic import]
api.py -> import email.message [HIGH RISK - may be called by framework/router/dynamic import]
api.py -> import typing [HIGH RISK - may be called by framework/router/dynamic import]
api.py -> import pathlib [HIGH RISK - may be called by framework/router/dynamic import]
api.py -> import argon2 [HIGH RISK - may be called by framework/router/dynamic import]
api.py -> import logging.handlers [HIGH RISK - may be called by framework/router/dynamic import]

api.py -> import sqlalchemy [HIGH RISK - may be called by framework/router/dynamic import]
 api.py -> import sqlalchemy.ext.declarative [HIGH RISK - may be called by framework/router/dynamic import]
 api.py -> import sqlalchemy.orm [HIGH RISK - may be called by framework/router/dynamic import]
 audit_log.py -> import db [MEDIUM RISK - verify references before removing]
 auth.py -> import email.mime.text [MEDIUM RISK - verify references before removing]
 auth.py -> import config [MEDIUM RISK - verify references before removing]
 auth.py -> import db [MEDIUM RISK - verify references before removing]
 auth.py -> import encryptor [MEDIUM RISK - verify references before removing]
 auth.py -> import logger [MEDIUM RISK - verify references before removing]
 backup.py -> import botocore.exceptions [MEDIUM RISK - verify references before removing]
 backup.py -> import dotenv [MEDIUM RISK - verify references before removing]
 db.py -> import config [MEDIUM RISK - verify references before removing]
 email_alert.py -> import email.mime.text [MEDIUM RISK - verify references before removing]
 email_alert.py -> import config [MEDIUM RISK - verify references before removing]
 encryptor.py -> import hashlib [MEDIUM RISK - verify references before removing]
 encryptor.py -> import bcrypt [MEDIUM RISK - verify references before removing]
 encryptor.py -> import hmac [MEDIUM RISK - verify references before removing]
 encryptor.py -> import secrets [MEDIUM RISK - verify references before removing]
 encryptor.py -> import email.message [MEDIUM RISK - verify references before removing]
 encryptor.py -> import typing [MEDIUM RISK - verify references before removing]
 encryptor.py -> import pathlib [MEDIUM RISK - verify references before removing]
 encryptor.py -> import argon2 [MEDIUM RISK - verify references before removing]
 encryptor.py -> import logging.handlers [MEDIUM RISK - verify references before removing]
 encryptor.py -> import sqlalchemy [MEDIUM RISK - verify references before removing]
 encryptor.py -> import sqlalchemy.ext.declarative [MEDIUM RISK - verify references before removing]
 encryptor.py -> import sqlalchemy.orm [MEDIUM RISK - verify references before removing]
 explain.py -> import numpy [MEDIUM RISK - verify references before removing]
 explain.py -> import pandas [MEDIUM RISK - verify references before removing]
 explain.py -> import io [MEDIUM RISK - verify references before removing]
 explain.py -> import matplotlib.pyplot [MEDIUM RISK - verify references before removing]
 explain.py -> import pathlib [MEDIUM RISK - verify references before removing]
 explain.py -> import typing [MEDIUM RISK - verify references before removing]
 gui.py -> import pandas [MEDIUM RISK - verify references before removing]
 gui.py -> import explainability [MEDIUM RISK - verify references before removing]
 logger.py -> import io [MEDIUM RISK - verify references before removing]
 logger.py -> import joblib [MEDIUM RISK - verify references before removing]
 logger.py -> import pandas [MEDIUM RISK - verify references before removing]
 logger.py -> import pathlib [MEDIUM RISK - verify references before removing]
 logger.py -> import explainability [MEDIUM RISK - verify references before removing]
 logger.py -> import typing [MEDIUM RISK - verify references before removing]
 logger.py -> import threading [MEDIUM RISK - verify references before removing]
 middleware.py -> import flask [HIGH RISK - may be called by framework/router/dynamic import]
 middleware.py -> import auth_utils [HIGH RISK - may be called by framework/router/dynamic import]
 permissions.py -> import logger [MEDIUM RISK - verify references before removing]
 permissions_dynamic.py -> import db [MEDIUM RISK - verify references before removing]
 permissions_dynamic.py -> import logger [MEDIUM RISK - verify references before removing]
 security.py -> import threading [MEDIUM RISK - verify references before removing]
 security.py -> import flask [MEDIUM RISK - verify references before removing]
 ... and 62 more

Possible unused test imports:

test\test_auth.py -> import pytest [LOW RISK - test function, used by test runner]
 test\test_auth.py -> import app.main [LOW RISK - test function, used by test runner]
 test\test_auth.py -> import app.auth [LOW RISK - test function, used by test runner]
 test\test_crypto.py -> import crypto.crypto [LOW RISK - test function, used by test runner]
 tests\test_auth.py -> import pytest [LOW RISK - test function, used by test runner]
 tests\test_auth.py -> import main [LOW RISK - test function, used by test runner]
 tests\test_dashboard_access.py -> import flask [LOW RISK - test function, used by test runner]
 tests\test_dashboard_access.py -> import flask.testing [LOW RISK - test function, used by test runner]
 tests\test_dashboard_access.py -> import dashboard [LOW RISK - test function, used by test runner]
 tests\test_dashboard_access.py -> import permissions_dynamic [LOW RISK - test function, used by test runner]
 tests\test_dashboard_access.py -> import db [LOW RISK - test function, used by test runner]
 tests\test_permissions.py -> import pytest [LOW RISK - test function, used by test runner]
 tests\test_permissions.py -> import permissions_dynamic [LOW RISK - test function, used by test runner]
 tests\test_permissions.py -> import db [LOW RISK - test function, used by test runner]

Possible unused docs/config imports:

config.py -> import dotenv [MEDIUM RISK - verify references before removing]

Recommendation:

Do not delete automatically. First search the full project, run tests, check dynamic imports/routes/config usage, and verify deployment scripts manually.

=====

Possible duplicated business logic blocks found:
Import-only duplicates and boilerplate were ignored.

Duplicate #1

Files:

api.py

encryptor.py

Code sample:

```
BLOCK_PERIOD = int(os.getenv("STR", NUM))
WINDOW_SECONDS = int(os.getenv("STR", NUM))
EMAIL_ALERTS_ENABLED = os.getenv("STR", "STR").lower() == "STR"
EMAIL_ADMIN = os.getenv("STR", "STR")
EMAIL_SENDER = os.getenv("STR", "STR")
def save_blacklist_ip(ip_address: str):
    session = Session()
    session.merge(BlacklistedIP(ip=ip_address, timestamp=time.time()))
```

Duplicate #2

Files:

api.py

encryptor.py

Code sample:

```
LOGIN_ATTEMPTS: Dict[str, list] = {}
MAX_ATTEMPTS = int(os.getenv("STR", NUM))
BLOCK_PERIOD = int(os.getenv("STR", NUM))
WINDOW_SECONDS = int(os.getenv("STR", NUM))
EMAIL_ALERTS_ENABLED = os.getenv("STR", "STR").lower() == "STR"
EMAIL_ADMIN = os.getenv("STR", "STR")
EMAIL_SENDER = os.getenv("STR", "STR")
def save_blacklist_ip(ip_address: str):
```

Duplicate #3

Files:

api.py

encryptor.py

Code sample:

```
Base.metadata.create_all(engine)
LOGIN_ATTEMPTS: Dict[str, list] = {}
MAX_ATTEMPTS = int(os.getenv("STR", NUM))
BLOCK_PERIOD = int(os.getenv("STR", NUM))
WINDOW_SECONDS = int(os.getenv("STR", NUM))
EMAIL_ALERTS_ENABLED = os.getenv("STR", "STR").lower() == "STR"
EMAIL_ADMIN = os.getenv("STR", "STR")
EMAIL_SENDER = os.getenv("STR", "STR")
```

Duplicate #4

Files:

api.py

encryptor.py

Code sample:

```
MAX_ATTEMPTS = int(os.getenv("STR", NUM))
BLOCK_PERIOD = int(os.getenv("STR", NUM))
WINDOW_SECONDS = int(os.getenv("STR", NUM))
EMAIL_ALERTS_ENABLED = os.getenv("STR", "STR").lower() == "STR"
EMAIL_ADMIN = os.getenv("STR", "STR")
EMAIL_SENDER = os.getenv("STR", "STR")
def save_blacklist_ip(ip_address: str):
    session = Session()
```

Duplicate #5

Files:

api.py

encryptor.py

Code sample:

```
WINDOW_SECONDS = int(os.getenv("STR", NUM))
EMAIL_ALERTS_ENABLED = os.getenv("STR", "STR").lower() == "STR"
EMAIL_ADMIN = os.getenv("STR", "STR")
EMAIL_SENDER = os.getenv("STR", "STR")
def save_blacklist_ip(ip_address: str):
    session = Session()
    session.merge(BlacklistedIP(ip=ip_address, timestamp=time.time()))
    session.commit()
```

Duplicate #6

Files:

api.py

encryptor.py

Code sample:

```
handler = RotatingFileHandler("STR", maxBytes=NUM * NUM * NUM, backupCount=NUM)
```

```
formatter = logging.Formatter("STR")
```

```
handler.setFormatter(formatter)
```

```
logger.addHandler(handler)
```

```
logger.setLevel(logging.INFO)
```

```
argon2_hasher = PasswordHasher()
```

```
PEPPER = os.getenv("STR", "STR")
```

```
DB_URL = os.getenv("STR", "sqlite:
```

Duplicate #7

Files:

api.py

encryptor.py

Code sample:

```
logger = logging.getLogger("STR")
```

```
handler = RotatingFileHandler("STR", maxBytes=NUM * NUM * NUM, backupCount=NUM)
```

```
formatter = logging.Formatter("STR")
```

```
handler.setFormatter(formatter)
```

```
logger.addHandler(handler)
```

```
logger.setLevel(logging.INFO)
```

```
argon2_hasher = PasswordHasher()
```

```
PEPPER = os.getenv("STR", "STR")
```

Duplicate #8

Files:

api.py

encryptor.py

Code sample:

```
EMAIL_ALERTS_ENABLED = os.getenv("STR", "STR").lower() == "STR"
```

```
EMAIL_ADMIN = os.getenv("STR", "STR")
```

```
EMAIL_SENDER = os.getenv("STR", "STR")
```

```
def save_blacklist_ip(ip_address: str):
```

```
    session = Session()
```

```
    session.merge(BlacklistedIP(ip=ip_address, timestamp=time.time()))
```

```
    session.commit()
```

```
    session.close()
```

Duplicate #9

Files:

api.py

encryptor.py

Code sample:

```
EMAIL_ADMIN = os.getenv("STR", "STR")
```

```
EMAIL_SENDER = os.getenv("STR", "STR")
```

```
def save_blacklist_ip(ip_address: str):
```

```
    session = Session()
```

```
    session.merge(BlacklistedIP(ip=ip_address, timestamp=time.time()))
```

```
    session.commit()
```

```
    session.close()
```

```
def is_ip_blocked(ip_address: str) -> bool:
```

Duplicate #10

Files:

api.py

encryptor.py

Code sample:

```
def save_blacklist_ip(ip_address: str):
```

```
    session = Session()
```

```
    session.merge(BlacklistedIP(ip=ip_address, timestamp=time.time()))
```

```
    session.commit()
```

```
    session.close()
```

```
def is_ip_blocked(ip_address: str) -> bool:
```

```
    session = Session()
```

```
    record = session.get(BlacklistedIP, ip_address)
```

7. PROJECT STRUCTURE

Project: CyberShield_AI_Enterprise_Hardened_Enhanced

Path: E:\stick\De facut\Cyber Security App\CyberShield_AI_Enterprise_Hardened_Enhanced

Files indexed: 78

Tech stack: CSS, Docker, Express, FastAPI, Flask, HTML, JWT, JavaScript, Node.js, Pytest, Python, React, SQLAlchemy, SQLite, Tailwind, TypeScript, Vite

Folders:

.github\workflows (1 files)

- .github\workflows\security.yml

admin (2 files)

- admin\admin.py
- admin\admin_permissions.html

audit_backups (3 files)

- audit_backups\decrypt.py
- audit_backups\export_audit.py
- audit_backups\export_audit_secure.py

backend (2 files)

- backend\audit_log.py
- backend\backup.py

backend\database (2 files)

- backend\database\file_scans.sql
- backend\database\init_db.py

backend\utils (1 files)

- backend\utils\crypto.py

components (1 files)

- components\AppLayout.tsx

crypto (2 files)

- crypto\crypto.py
- crypto\key_rotation_scheduler.py

dashboard (2 files)

- dashboard\dashboard.html
- dashboard\dashboard.py

docs (2 files)

- docs\DEPLOYMENT_GUIDE.md
- docs\SECURITY_OVERVIEW.md

file-scan-log-app (5 files)

- file-scan-log-app\package-lock.json
- file-scan-log-app\package.json
- file-scan-log-app\postcss.config.js
- file-scan-log-app\tailwind.config.js
- file-scan-log-app\vite.config.ts

file-scan-log-app\src (4 files)

- file-scan-log-app\src\FileScanLog.tsx
- file-scan-log-app\src\index.css
- file-scan-log-app\src\main.tsx
- file-scan-log-app\src\ScanForm.tsx

frontend (1 files)

- frontend\README.md

models (1 files)

- models\user.py

root (29 files)

- aes_cipher.py
- api.py
- auditUI.py
- audit_log.py
- auth.py
- backup.py
- cloudflare_firewall_template.json
- cloudflare_waf_rules.md
- config.py
- db.py
- deploy_to_render.sh
- docker-compose.yml
- email_alert.py

- encryptor.py
- explain.py
- gui.py
- logger.py
- middleware.py
- package-lock.json
- package.json

routes (4 files)

- routes\admin.py
- routes\auth.py
- routes\dashboard.py
- routes__init__.py

schemas (3 files)

- schemas\admin.py
- schemas\auth.py
- schemas\dashboard.py

templates (1 files)

- templates\dashboard.html

templates\Dashboard Audit (2 files)

- templates\Dashboard Audit\audit_dashboard.html
- templates\Dashboard Audit\audit_dashboard.py

test (3 files)

- test\security_scan.sh
- test\test_auth.py
- test\test_crypto.py

tests (3 files)

- tests\test_auth.py
- tests\test_dashboard_access.py
- tests\test_permissions.py

utils (4 files)

- utils\auth.py
- utils\auth_utils.py
- utils\logger.py
- utils\token.py

8. IMPROVEMENT ROADMAP

All configured models failed.

Last error: 404 Client Error: Not Found for url: http://localhost:11434/api/chat

9. PROJECT DOCUMENTATION

All configured models failed.

Last error: 404 Client Error: Not Found for url: http://localhost:11434/api/chat